A report on

"OOP's Using JAVA Lab"

Submitted in partial fulfillment of the requirements for the award of degree

BACHELOR OF ENGINEERING

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

By

PRINCE KHATAVLIA (25BTRCL196)

Under the guidance of

B LAVANYA

Assistant Professor

Department of CSE - Internet of Things

2025-2026

C E R T I F I C A T E

This is to certify that the report entitled "***OOP's Using JAVA Lab***" is a Bonafide work carried out by ***PRINCE KHATAVLIA (25BTRCL196)*** in partial fulfillment for the award of degree of Bachelor of Technology in "Artificial Intelligence and Machine Learning" from Jain - deemed to be University, Bangalore during the year 2025-2026. The report has been approved as it satisfies the academic requirements in respect of laboratories prescribed for the Bachelor of Technology Degree.

Signature of the Faculty

B Lavanya

Assistant professor

Department - CSE- Internet of Things

INDEX

Sl. No	Name	Pg. No	Signature
1	Loan Eligibility Program using If–Else	4	
2	Scholarship Eligibility using Nested If–Else	5	
3	Display Roll Numbers using For Loop	6	
4	Student Grade Evaluation using If–Else, Switch and Loop	7-8	
5	Student Class to Store and Display Details	9	
6	Circle Class using Constructor Overloading	10	
7	Method Overloading for Area Calculation	11	
8	Use of Static and Final Keywords in Java	12	
9	Use of this Keyword to Resolve Ambiguity	13	
10	Inheritance in Robotics System (Single & Multilevel)	14	
11	Employee Management System using Inheritance	15	
12	Runtime Polymorphism using Method Overriding	16	

1. Loan Eligibility Program

Question: A bank wants to develop a simple program to check whether a customer is eligible for a loan. If salary ≥ ₹30,000 → Eligible, otherwise not eligible.

Code:

```
import java.util.Scanner;
public class LoanEligibility {
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.print("Enter salary: ");
		int salary = sc.nextInt();
		if (salary >= 30000) {
			System.out.println("Eligible for Loan");
		}
		else {
			System.out.println("Not Eligible for Loan");
		}
	}
}
```

Output:

```
Enter salary: 35000
Eligible for Loan

Process finished with exit code 0
```

2. Scholarship Eligibility Program

Question: Determine scholarship based on marks and family income.

Code:

```
import java.util.Scanner;
public class Scholarship {
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.print("Enter marks: ");
		int marks = sc.nextInt();
		System.out.print("Enter family income: ");
		int income = sc.nextInt();
		if (marks >= 75) {
			if (income <= 200000) {
				System.out.println("Full Scholarship");
			}
			else if (income <= 500000) {
				System.out.println("Partial Scholarship");
			}
			else {
				System.out.println("Not Eligible for Scholarship");
			}
		}
		else {
			System.out.println("Not Eligible");
		}
	}
}
```

Output

```
Enter marks: 80
Enter family income: 300000
Partial Scholarship

Process finished with exit code 0
```

3. Roll Number Display

Question: Print roll numbers from 1 to 10.

Code:

```
public class RollNumbers {
	public static void main(String[] args) {
	for (int i = 1; i <= 10; i++) {
		System.out.println("Roll No: " + i);
	}
	}
}
```

Output:

```
Roll No: 1
Roll No: 2
Roll No: 3
Roll No: 4
Roll No: 5
Roll No: 6
Roll No: 7
Roll No: 8
Roll No: 9
Roll No: 10

Process finished with exit code 0
```

4. Student Grade Evaluation

Question: Assign grades and use switch + loop.

Code:

```
import java.util.Scanner;
public class GradeSystem {
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		char choice;
		do {
			System.out.print("Enter marks: ");
			int marks = sc.nextInt();
			char grade;
			if (marks >= 90)
				grade = 'A';
			else if (marks >= 75)
				grade = 'B';
			else if (marks >= 50)
				grade = 'C';
			else
				grade = 'F';

			switch (grade) {
				case 'A':
					System.out.println("Excellent");
					break;
				case 'B':
					System.out.println("Good");
					break;
				case 'C':
					System.out.println("Average");
					break;
				default:
					System.out.println("Fail");
			}
			System.out.print("Continue? (y/n): ");
			choice = sc.next().charAt(0);
```

```
                } while (choice == 'y');
        }
}
```

Output:

```
Enter marks: 85
Good
Continue? (y/n): n

Process finished with exit code 0
```

5. Student Class Program

Question: A college needs a program to store and display student details such as name, roll number, and marks.

Code:

```
class Student {
	String name;
	int roll;
	int marks;
	void display() {
		System.out.println(name + " " + roll + " " + marks);
	}
	public static void main(String[] args) {
		Student s = new Student();
		s.name = "Rahul";
		s.roll = 1;
		s.marks = 90;
		s.display();
	}
}
```

Output:

```
Rahul 1 90

Process finished with exit code 0
```

6. Define a class Circle with Overloaded Constructors

Question: A graphics application needs to create circles in two ways:

1. With a **default radius**
2. With a **user-defined radius**

To support both cases, constructor overloading is required.

Code:

```
class Circle {
	double radius;
	Circle() {
		radius = 1.0;
	}
	Circle(double r) {
		radius = r;
	}
	void area() {
		System.out.println("Area = " + (3.14 * radius * radius));
	}
	public static void main(String[] args) {
		Circle c1 = new Circle();
		Circle c2 = new Circle(5);
		c1.area(); c2.area();
	}
}
```

Output:

```
Area = 3.14
Area = 78.5

Process finished with exit code 0
```

7. Method Overloading

Question: A math utility program must calculate the area of different shapes

Code:

```
class Area {
	void calculateArea(double r) {
		System.out.println("Circle Area: " + (3.14 * r * r));
	}
	void calculateArea(int l, int b) {
		System.out.println("Rectangle Area: " + (l * b));
	}
	void calculateArea(int s) {
		System.out.println("Square Area: " + (s * s));
	}
	public static void main(String[] args) {
		Area a = new Area();
		a.calculateArea(2.5);
		a.calculateArea(4,5);
		a.calculateArea(3);
	}
}
```

Output:

```
Circle Area: 19.625
Rectangle Area: 20
Square Area: 9

Process finished with exit code 0
```

8. Static and Final

Question: A college wants to store the college name (constant) and count how many students are created.

Code:

```
class College {
        final String COLLEGE_NAME = "ABC College";
        static int count = 0; College() {
                count++;
        }
        void display() {
                System.out.println(COLLEGE_NAME + " Student Count: " + count);
        }
        public static void main(String[] args) {
        College c1 = new College();
        College c2 = new College();
        c1.display(); c2.display();
        }
}
```

Output:

```
ABC College Student Count: 2
ABC College Student Count: 2

Process finished with exit code 0
```

9. this Keyword

Question: A pet management system stores dog details.

Code:

```
class Dog {
	String name;
	Dog(String name) {
		this.name = name;
	}
	void display() {
		System.out.println("Dog Name: " + name);
	}
	public static void main(String[] args) {
		Dog d = new Dog("Tommy");
		d.display();
	}
}
```

Output:

```
Dog Name: Tommy

Process finished with exit code 0
```

10. Inheritance (Robot)

Code:

```
class Robot {
	void move() {
		System.out.println("Robot moves");
	}
}
class ServiceRobot extends Robot {
	void assist() {
		System.out.println("Assists humans");
	}
}
class AdvancedRobot extends ServiceRobot {
	void aiTask() {
		System.out.println("Performs AI tasks");
	}
	public static void main(String[] args) {
		AdvancedRobot r = new AdvancedRobot();
		r.move();
		r.assist();
		r.aiTask();
	}
}
```

Output:

```
Robot moves
Assists humans
Performs AI tasks

Process finished with exit code 0
```

11. Employee Management

Code:

```
class Employee {
	String name; int salary;
}
class Manager extends Employee {
	String department;
}
class SeniorManager extends Manager {
	int bonus;
	void display() {
		System.out.println(name + " " + salary + " " + department + " " + bonus);
	}
	public static void main(String[] args) {
		SeniorManager s = new SeniorManager();
		s.name = "Raj";
		s.salary = 50000;
		s.department = "IT";
		s.bonus = 10000;
		s.display();
	}
}
```

Output:

```
Raj 50000 IT 10000

Process finished with exit code 0
```

12. Runtime Polymorphism

Code:

```
class Employee {
	void calculateSalary() {
		System.out.println("Base Salary");
	}
}
class FullTimeEmployee extends Employee {
	void calculateSalary() {
		System.out.println("Fixed Salary: 50000");
	}
}
class PartTimeEmployee extends Employee {
	void calculateSalary() {
		System.out.println("Hourly Salary: 200/hr");
	}
	public static void main(String[] args) {
		Employee e;
		e = new FullTimeEmployee();
		e.calculateSalary();
		e = new PartTimeEmployee();
		e.calculateSalary();
	}
}
```

Output:

```
Fixed Salary: 50000
Hourly Salary: 200/hr

Process finished with exit code 0
```